Assignment 3. Graph Algorithms

Design and Analysis of Algorithms | Campus festival | Practical work: 50 points

1. Implement the algorithms

Create your repository using **Use this template**. Choose Java 17, Python 3.12+, C++17 or C# (.NET 8); set Java, Python, Cpp or CSharp in **submission.json**. Complete the six methods in **src/**. Implement the graph algorithms yourself; standard collections, sorting and priority queues are allowed. Input/output adapters are provided.

Method	Required result
dsu	Use path compression and union by size/rank. For each pair in operations, return [components, largest] in states after the union.
kruskal **prim**	Write two separate minimum spanning forest algorithms for undirected weighted edges. Return cost, original edge_ids and components, including isolates. Prim must restart in each component.
scc	Use Kosaraju: two DFS passes and a transpose. Return vertex labels in component and every distinct cross-component pair in condensation.
topo	Use three-state DFS on a directed graph. Return acyclic and order. If any directed cycle exists, return false and an empty order.
dag	Check the whole graph for cycles, then relax edges in topological order from source. Return acyclic, distance and parent_edge. For a cycle, return false and two empty arrays. Add a route-reconstruction helper.

2. Keep the interface and limits

Use UTF-8 JSON Lines: one request and one result per line; no other stdout output. Reset state between requests and preserve ping. Vertices are 0..n-1; edge IDs are original positions in edges. Rows: DSU operations [u,v]; SCC/topo edges [u,v]; weighted edges [u,v,w]. Every request includes task and n; dag also includes source.

Limits: $n \le 100000$; m or union operations ≤ 250000; $-10^9 \le w \le 10^9$. Empty graphs are valid except dag ($n \ge 1$). Use 64-bit totals or Python integers. Handle loops, parallel edges, negative weights, isolates and deep graphs. In DAG results, unreachable distance=null and parent_edge=-1; source distance=0 and parent_edge=-1. Other parents are original edge IDs. Accept any valid optimal forest, SCC labels, topological order or shortest-path witness.

3. Test and explain your work

Run **python tools/run.py --setup** and **python tools/run.py --check**. The 36 public cases report pass/fail; they do not calculate or upload grades. Add your own tests in **tests/**: inputs, justified expected answers, actual outputs and commands. Cover boundaries, full limits, deep chains, repeated mixed requests and cycles unreachable from the source. Check non-unique answers by validity, including an alternative valid answer and an invalid witness.

Create **ANALYSIS.md in English**: steps, invariants, correctness, boundary cases and time/memory complexity for each method; state reset and deep DFS. Show route-helper calls for a reachable target, source, unreachable target and tied/parallel edges. Compare Kruskal and Prim on the same graphs at three sizes: at least three timings per method per size and medians. Record inputs/seed, commands, environment, timer, warm-up and timing boundary; exclude compilation. Acknowledge sources and AI assistance according to the syllabus.

4. Submit and check the assessment

Your commit must include source code, submission.json, tests/ and ANALYSIS.md with build/run/test commands. Pass Public checks in your copy. In Moodle **Online text**, submit the exact commit URL, then click **Submit assignment**; you may add the Actions link. Give me access to a private repository. Follow the Moodle deadline and approved extensions.

Practical points: DSU 8; Kruskal 7; Prim 7; SCC 10; topo 8; DAG 10 (total 50). The separate quiz contributes 50 points. The methods, helper, tests, analysis and benchmark are required. Full JSON contracts, examples, complexity targets and the 18-group rubric are in the repository; use them when implementing and testing.

Template and full specification (English / Russian)